



Workspace Management Implementation Plan

Product context

Orviahub will contain multiple applications:

```
orviahub.com
accounts.orviahub.com
app.orviahub.com
inventory.orviahub.com
taskmanagement.orviahub.com
api.orviahub.com
```

Organizations will **not** use subdomains. In the product, an organization is represented as a **workspace**.

Example:

```
Workspace: Code X Stores
Workspace ID: ws_123
Workspace slug: code-x-stores
Application: Inventory
Current user role: Owner
```

The workspace slug can be used for display, internal references, invitations, or optional path URLs, but never as:

```
code-x-stores.orviahub.com
```

Only applications use subdomains.

Main objective

Build a reusable workspace-management system that supports:

- Workspace creation.
- Workspace onboarding.
- Workspace switching.
- Workspace membership.
- Invitations.

- Roles and permissions.
- Product access.
- Workspace settings.
- Branches and locations.
- Subscription and usage limits.
- Ownership transfer.
- Workspace suspension.
- Workspace deletion.
- Audit logs.
- Notifications.
- Future applications.

The system must work for Inventory first, but its design should support Task Management, Gym Management, Bookings, CRM, and future products without rebuilding workspace functionality.

1. Workspace concepts

Workspace

A workspace is the business or team environment where users operate.

Examples:

- Code X Stores.
- Orviohub Team.
- Enugu Fitness Club.
- ABC Construction Projects.

A workspace contains:

- Basic profile.
- Members.
- Roles.
- Applications.
- Subscription.
- Settings.
- Branches or locations.
- Product-specific data.

User

A person with one Orviohub account.

A user can belong to multiple workspaces.

Workspace membership

Defines the relationship between a user and a workspace.

A membership contains:

- User.
- Workspace.
- Status.
- Role.
- Permissions.
- Invitation information.
- Joined date.
- Suspension state.

Product entitlement

Defines whether a workspace can use an application.

For example:

```
Code X Stores
- Inventory: Active
- Task Management: Inactive
- CRM: Trial
```

Product membership

Defines what a user can do inside a particular product.

For example:

```
Code X
- Workspace: Code X Stores
- Product: Inventory
- Role: Owner
```

```
Mary
- Workspace: Code X Stores
- Product: Inventory
- Role: Sales Attendant
```

```
Peter
```

- Workspace: Code X Stores
- Product: Task Management
- Role: Member

Branch

A physical or logical location inside a workspace.

For Inventory:

- Main Store.
- Abak Road Branch.
- Warehouse.
- Online Store.

For Task Management, a workspace may not need branches, but the shared model can support them as optional locations.

2. User journey overview

New user creates a workspace

```
orviahub.com
  ↓
Choose product
  ↓
Sign up or sign in
  ↓
Complete global account profile
  ↓
Choose "Create a workspace"
  ↓
Enter workspace details
  ↓
Select product
  ↓
Complete product onboarding
  ↓
Invite members
  ↓
Enter product dashboard
```

Existing user selects a workspace

```
inventory.orviahub.com
  ↓
Authenticate through accounts.orviahub.com
  ↓
Fetch user workspaces
```

↓
Show Inventory-enabled workspaces
↓
User selects workspace
↓
Resolve membership and permissions
↓
Enter selected workspace dashboard

User creates another workspace

app.orviohub.com
↓
Workspace switcher
↓
Create workspace
↓
Enter details
↓
Select products
↓
Complete onboarding
↓
New workspace becomes active

User accepts invitation

Invitation email
↓
Open invitation link
↓
Authenticate or create account
↓
Review workspace, product, and role
↓
Accept invitation
↓
Membership activated
↓
Redirect to product dashboard

User switches workspace

Product dashboard
↓
Open workspace switcher
↓
Select another workspace
↓
Refresh authorization context
↓

```
Load selected workspace data
↓
Remain on equivalent product page
```

Example:

```
inventory.orviohub.com/products
```

After switching, the user still sees the Products page, but for the new workspace.

3. Workspace lifecycle

A workspace should have a controlled lifecycle.

Workspace statuses

```
creating
active
setup_incomplete
trial
past_due
suspended
archived
deleting
deleted
```

Status meanings

Creating

Workspace record has been created but initialization is not complete.

Setup incomplete

Workspace exists, but required product onboarding is unfinished.

Active

Workspace is operational.

Trial

Workspace is using a trial entitlement.

Past due

Subscription payment failed or is overdue.

Suspended

Access is restricted due to billing, policy, or administrative action.

Archived

Workspace is no longer active but retained according to policy.

Deleting

Workspace deletion has been confirmed and is being processed.

Deleted

Workspace is no longer available for normal access.

4. Workspace creation journey

Entry points

A user can create a workspace from:

- Global onboarding.
- `app.orviohub.com`.
- Workspace switcher.
- Product activation screen.
- Invitation rejection or “create your own workspace” flow.

Screen 1: Create workspace

URL:

```
accounts.orviohub.com/workspaces/new
```

Fields

- Workspace name.
- Workspace type.
- Country.
- State.
- City.

- Time zone.
- Default currency.
- Phone number.
- Logo, optional.

Workspace types

Initial options:

- Business.
- Team.
- Personal.
- Club or community.
- School or institution.
- Other.

For Inventory, “Business” should be selected by default.

Screen 2: Workspace type configuration

Depending on the selected type, ask additional questions.

Business

- Business category.
- Number of locations.
- Number of employees.
- Current record-keeping method.

Team

- Team size.
- Primary work type.
- Main collaboration goal.

Personal

- Main use case.
- Private or shared workspace.

These questions should personalize onboarding but must not create rigid product restrictions.

Screen 3: Confirm workspace

Display:

- Workspace name.
- Workspace type.
- Country.
- Currency.
- Owner.
- Selected applications.

Actions:

- Create workspace.
- Go back.

Backend transaction

Creating a workspace should create:

1. Workspace record.
2. Owner membership.
3. Default workspace settings.
4. Default role set.
5. Product entitlements for selected applications.
6. Onboarding flow records.
7. Audit event.
8. Welcome notification.

The operation should be idempotent so retrying the request does not create duplicate workspaces.

5. Workspace selection journey

Workspace selector requirements

The selector should display:

- Workspace logo.
- Workspace name.
- Workspace type.
- Active application.
- User's role.

- Workspace status.
- Recently used workspaces.
- Search.
- Create workspace.
- Join workspace.

Product-aware workspace filtering

When a user visits Inventory, show only workspaces where:

- The user is an active member.
- Inventory is enabled.
- Workspace is not suspended.
- The user has product access.

When a user visits Task Management, show the equivalent Task Management workspaces.

Screen

```
app.orviohub.com/workspaces
```

Empty state

If the user has no workspace:

You do not belong to a workspace yet. Create one or accept an invitation to get started.

Actions:

- Create workspace.
- Join with invitation.

Default workspace

Allow the user to set a default workspace per application.

Example:

```
Inventory default: Code X Stores  
Task Management default: Orviohub Team
```

6. Workspace switching journey

Requirements

Users can switch workspaces from:

- Application header.
- Application launcher.
- Account center.
- Workspace selector.

Switching process

1. User opens switcher.
2. System displays accessible workspaces.
3. User selects a workspace.
4. Frontend sends workspace-switch request.
5. Backend verifies membership and product access.
6. Backend returns a new workspace context.
7. Frontend clears current workspace data.
8. Frontend loads the selected workspace.
9. UI displays the new workspace name.
10. Audit event is created where appropriate.

Important security rule

Do not switch workspaces by simply changing a client-side `workspaceId`.

The backend must verify:

```
authenticated user
+ active membership
+ product access
+ workspace status
```

OWASP recommends establishing tenant context early, binding it to the authenticated session, and validating resource ownership at the data-access layer rather than trusting client-supplied tenant IDs. ^[1]

URL behavior

You can use either:

```
inventory.orviohub.com/dashboard
```

with active workspace stored in session/context, or:

```
inventory.orviohub.com/workspaces/ws_123/dashboard
```

For the MVP, I recommend:

```
inventory.orviohub.com/dashboard
```

while the backend resolves the active workspace from a secure session or explicit validated request context.

If you use workspace IDs in URLs later, use non-guessable IDs and still validate membership on every request.

7. Workspace invitation journey

Invitation creation

An authorized owner or manager opens:

```
inventory.orviohub.com/settings/members
```

They select:

- Invite member.
- Email or phone.
- Product.
- Role.
- Branch access.
- Optional message.

Example:

```
Email: staff@example.com  
Product: Inventory  
Role: Sales Attendant  
Branch: Main Store
```

Invitation confirmation

Before sending, show:

- Invitee.
- Workspace name.
- Product.
- Role.
- Branch.
- Invitation expiration.
- Sender.

Invitation delivery

Send:

- Email invitation.
- In-app invitation if the user already has an account.
- Optional SMS or WhatsApp notification later.

Invitation screen

URL:

```
accounts.orviohub.com/invitations/:token
```

Display:

- Workspace logo.
- Workspace name.
- Inviting person.
- Product.
- Role.
- Branch access.
- Invitation expiry.
- Accept.
- Decline.

New invitee

If the invitee has no account:

1. Select Create account.
2. Complete authentication.
3. Verify email if necessary.
4. Return to invitation.
5. Review invitation.
6. Accept.

Existing invitee

If the invitee already has an account:

1. Sign in.
2. Return to invitation.
3. Review invitation.
4. Accept.
5. Become an active member.

Invitation states

```
pending
accepted
declined
expired
revoked
cancelled
```

Invitation rules

- Token must be single-use.
- Token must be hashed in the database.
- Invitation must expire.
- Sender can revoke invitation.
- Invitee can decline.
- Invitation cannot grant access before acceptance.
- Role must be visible before acceptance.
- Acceptance must be audited.
- Duplicate active invitations should be prevented or reused.

8. Membership management

Member list screen

URL:

```
inventory.orviohub.com/settings/members
```

Display:

- Name.
- Email.
- Profile image.
- Role.
- Product access.
- Branch access.
- Membership status.
- Last active.
- Joined date.
- Actions.

Member actions

Depending on permission:

- View member.
- Change role.
- Change branch access.
- Grant product access.
- Revoke product access.
- Suspend member.
- Remove member.
- Resend invitation.
- View activity.

Membership statuses

```
invited  
active  
suspended
```

removed
left

Remove member flow

1. Authorized user selects Remove.
2. System explains effects.
3. User confirms.
4. Membership changes to removed.
5. Product access is revoked.
6. Existing sessions are invalidated.
7. Member receives notification.
8. Audit event is recorded.

Do not delete historical transactions created by the removed member.

9. Roles and permissions

Role model

Use platform-level roles plus product-level roles.

Platform roles

- Workspace Owner.
- Workspace Admin.
- Workspace Member.
- Workspace Viewer.

Inventory roles

- Inventory Owner.
- Inventory Manager.
- Sales Attendant.
- Stock Manager.
- Accountant.
- Inventory Viewer.

Task Management roles

- Task Owner.
- Project Manager.
- Contributor.
- Viewer.

A user can have one platform role and one or more product roles.

Permission model

Permissions should be explicit.

Example:

```
workspace.view
workspace.update
workspace.manage_members
workspace.manage_roles
workspace.manage_products
workspace.manage_billing
workspace.delete

inventory.view
inventory.sell
inventory.manage_products
inventory.receive_stock
inventory.adjust_stock
inventory.view_reports
inventory.export_data
```

Permission evaluation

The backend should resolve:

```
user membership
+ workspace role
+ product membership
+ product role
+ explicit permission
+ branch scope
```

Branch-scoped permissions

A user may be restricted to:

- All branches.
- Main Store only.
- Warehouse only.

- A selected group of branches.

This is important for Inventory.

Owner protection

The workspace must always have at least one owner.

Rules:

- Owner cannot remove themselves if they are the only owner.
- Ownership transfer requires explicit confirmation.
- Transfer requires the new owner to be an active member.
- The old owner becomes Admin or another selected role.
- Ownership transfer creates high-severity audit event.
- Notify both users.

10. Workspace settings

General settings

- Workspace name.
- Workspace type.
- Logo.
- Country.
- State.
- City.
- Time zone.
- Currency.
- Contact details.
- Workspace slug.
- Status.

Member settings

- Members.
- Roles.
- Invitations.
- Access requests.
- Branch access.

Product settings

- Enabled applications.
- Product roles.
- Product configuration.
- Product trial status.
- Product usage.

Billing settings

- Current plan.
- Usage.
- Payment method.
- Billing contact.
- Invoices.
- Upgrade.
- Downgrade.
- Cancel subscription.

Security settings

- Active sessions.
- Support access.
- Workspace security activity.
- Login policy in future.
- MFA requirement in future.

Data settings

- Export workspace data.
- Archive workspace.
- Delete workspace.
- Retention information.

11. Product activation

Product activation journey

1. User opens application launcher.
2. User selects a product.
3. System checks workspace entitlement.
4. If inactive, show product overview.
5. User selects Start trial or Activate.
6. System creates workspaceProducts record.
7. System creates product onboarding flow.
8. System sends activation notification.
9. Redirect to product onboarding.

Product activation states

```
available  
trial  
active  
past_due  
suspended  
cancelled
```

Product access page

Display:

- Product name.
- Description.
- Features.
- Current status.
- Trial period.
- Plan.
- Usage limits.
- Activate or upgrade action.
- Support link.

12. Workspace deletion and archiving

Workspace archive

Archiving is reversible.

Use when:

- Business is temporarily closed.
- Project has ended.
- Workspace is no longer active.
- Owner wants to preserve records.

Effects:

- Members cannot perform normal actions.
- Product access becomes read-only or disabled.
- Billing may stop.
- Data is preserved.
- Workspace can be restored by owner or platform admin.

Workspace deletion

Deletion is destructive and must be carefully controlled.

Flow

1. Owner opens workspace settings.
2. Selects Delete workspace.
3. System explains:
 - Products affected.
 - Members affected.
 - Data retention.
 - Billing consequences.
4. Owner enters workspace name.
5. Owner re-authenticates.
6. Owner confirms.
7. Workspace enters deleting.
8. Active sessions are revoked.
9. Product access is disabled.
10. Members are notified.
11. Data deletion or anonymization process runs.
12. Audit record is retained according to policy.

Rules

- Only owner can request deletion.
- Require recent authentication.
- Require explicit confirmation.
- Add a grace period where appropriate.
- Support cancellation during grace period.
- Do not delete required financial or audit records without policy review.
- Document deletion behavior clearly.

13. Ownership transfer

Flow

1. Owner opens Members.
2. Selects active member.
3. Selects Transfer ownership.
4. System explains consequences.
5. Owner re-authenticates.
6. New owner confirms or accepts.
7. Ownership changes.
8. Old owner receives new role.
9. Sessions may be refreshed.
10. Both users receive notifications.
11. High-severity audit event is written.

Acceptance criteria

- Cannot transfer to a removed or suspended member.
- Cannot transfer without confirmation.
- Cannot leave workspace without an owner.
- Transfer cannot be completed using only frontend state.
- All related product ownership permissions are recalculated.

14. Workspace data isolation

Every workspace-owned record must include workspace context.

For application data:

```
workspaceId
productKey
branchId, where applicable
```

API authorization

For every request:

1. Authenticate user.
2. Resolve workspace.
3. Verify active membership.
4. Verify product entitlement.
5. Verify role and permission.
6. Query using workspace-scoped index.
7. Confirm the returned resource belongs to the workspace.
8. Log suspicious cross-workspace attempts.

Convex indexes

Convex recommends indexes for efficient and predictable queries, using `.index()` in the schema and `.withIndex()` in queries. [\[2\]](#) [\[3\]](#)

Recommended indexes:

```
workspaceMemberships.by_user
workspaceMemberships.by_workspace
workspaceMemberships.by_workspace_user
workspaceMemberships.by_user_status

workspaceProducts.by_workspace
workspaceProducts.by_workspace_product
workspaceProducts.by_product_status

productMemberships.by_workspace
productMemberships.by_workspace_user
productMemberships.by_workspace_product
productMemberships.by_user_product

invitations.by_token_hash
invitations.by_workspace
invitations.by_email_status
invitations.by_invited_user
```

```
onboardingFlows.by_user
onboardingFlows.by_workspace_product

auditLogs.by_workspace
auditLogs.by_actor
auditLogs.by_workspace_created
notifications.by_user
notifications.by_workspace
```

Use composite indexes for common access patterns:

```
workspaceId + userId
workspaceId + productKey
workspaceId + status
userId + status
```

Do not rely on fetching a broad table and filtering in application code.

15. Convex workspace schema

workspaces

```
workspaces: defineTable({
  name: v.string(),
  slug: v.string(),
  type: v.string(),
  ownerId: v.id("users"),
  country: v.string(),
  state: v.optional(v.string()),
  city: v.optional(v.string()),
  timezone: v.string(),
  currency: v.string(),
  logoUrl: v.optional(v.string()),
  status: v.string(),
  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_owner", ["ownerId"])
.index("by_slug", ["slug"])
.index("by_status", ["status"])
```

workspaceMemberships

```
workspaceMemberships: defineTable({
  workspaceId: v.id("workspaces"),
  userId: v.id("users"),
  status: v.string(),
  role: v.string(),
  invitedBy: v.optional(v.id("users")),
```



```

    invitedAt: v.optional(v.number()),
    acceptedAt: v.optional(v.number()),
    suspendedAt: v.optional(v.number()),
    createdAt: v.number(),
    updatedAt: v.number(),
  })
  .index("by_workspace", ["workspaceId"])
  .index("by_user", ["userId"])
  .index("by_workspace_user", ["workspaceId", "userId"])
  .index("by_user_status", ["userId", "status"])

```

workspaceProducts

```

workspaceProducts: defineTable({
  workspaceId: v.id("workspaces"),
  productKey: v.string(),
  status: v.string(),
  planId: v.optional(v.string()),
  trialStartedAt: v.optional(v.number()),
  trialEndsAt: v.optional(v.number()),
  activatedBy: v.id("users"),
  activatedAt: v.number(),
  suspendedAt: v.optional(v.number()),
})
  .index("by_workspace", ["workspaceId"])
  .index("by_workspace_product", ["workspaceId", "productKey"])
  .index("by_product_status", ["productKey", "status"])

```

productMemberships

```

productMemberships: defineTable({
  workspaceId: v.id("workspaces"),
  userId: v.id("users"),
  productKey: v.string(),
  role: v.string(),
  permissions: v.array(v.string()),
  status: v.string(),
  createdAt: v.number(),
  updatedAt: v.number(),
})
  .index("by_workspace", ["workspaceId"])
  .index("by_workspace_user", ["workspaceId", "userId"])
  .index("by_workspace_product", ["workspaceId", "productKey"])
  .index("by_user_product", ["userId", "productKey"])

```

workspaceInvitations

```

workspaceInvitations: defineTable({
  workspaceId: v.id("workspaces"),
  productKey: v.optional(v.string()),
  email: v.string(),

```

```

    emailNormalized: v.string(),
    inviteeUserId: v.optional(v.id("users")),
    role: v.string(),
    branchIds: v.optional(v.array(v.id("branches"))),
    tokenHash: v.string(),
    status: v.string(),
    invitedBy: v.id("users"),
    expiresAt: v.number(),
    acceptedAt: v.optional(v.number()),
    declinedAt: v.optional(v.number()),
    revokedAt: v.optional(v.number()),
    createdAt: v.number(),
  })
  .index("by_token_hash", ["tokenHash"])
  .index("by_workspace", ["workspaceId"])
  .index("by_email_status", ["emailNormalized", "status"])
  .index("by_invitee", ["inviteeUserId"])

```

branches

```

branches: defineTable({
  workspaceId: v.id("workspaces"),
  name: v.string(),
  code: v.optional(v.string()),
  address: v.optional(v.string()),
  phone: v.optional(v.string()),
  status: v.string(),
  createdAt: v.number(),
  updatedAt: v.number(),
})
  .index("by_workspace", ["workspaceId"])
  .index("by_workspace_status", ["workspaceId", "status"])

```

workspaceAuditLogs

```

workspaceAuditLogs: defineTable({
  workspaceId: v.id("workspaces"),
  actorUserId: v.optional(v.id("users")),
  eventType: v.string(),
  entityType: v.string(),
  entityId: v.optional(v.string()),
  severity: v.string(),
  metadata: v.optional(v.any()),
  requestId: v.optional(v.string()),
  ipAddress: v.optional(v.string()),
  userAgent: v.optional(v.string()),
  createdAt: v.number(),
})
  .index("by_workspace", ["workspaceId"])
  .index("by_workspace_created", ["workspaceId", "createdAt"])
  .index("by_actor", ["actorUserId"])

```

16. Fastify implementation structure

```
src/  
├── modules/  
│   ├── workspaces/  
│   │   ├── workspace.routes.ts  
│   │   ├── workspace.controller.ts  
│   │   ├── workspace.service.ts  
│   │   ├── workspace.repository.ts  
│   │   ├── workspace.schemas.ts  
│   │   └── workspace.permissions.ts  
│   ├── memberships/  
│   ├── invitations/  
│   ├── branches/  
│   ├── products/  
│   ├── entitlements/  
│   ├── onboarding/  
│   ├── notifications/  
│   └── audit/  
├── plugins/  
│   ├── auth.ts  
│   ├── convex.ts  
│   ├── request-context.ts  
│   ├── rate-limit.ts  
│   └── logger.ts  
├── hooks/  
│   ├── authenticate.ts  
│   ├── resolve-workspace.ts  
│   ├── require-membership.ts  
│   ├── require-product-access.ts  
│   └── require-permission.ts  
└── server.ts
```

Request lifecycle

```
Request  
↓  
Assign request ID  
↓  
Authenticate session  
↓  
Resolve workspace  
↓  
Verify membership  
↓  
Verify product access  
↓  
Verify permission  
↓  
Validate input  
↓  
Execute service  
↓
```

```
Write audit event
↓
Send notification if necessary
↓
Return response
```

Fastify's plugin and hook model is appropriate for separating authentication and authorization checks into reusable route-level or plugin-level behavior. [\[4\]](#) [\[5\]](#)

17. Workspace API endpoints

Workspace endpoints

```
GET    /v1/workspaces
POST   /v1/workspaces
GET    /v1/workspaces/:workspaceId
PATCH /v1/workspaces/:workspaceId
DELETE /v1/workspaces/:workspaceId
POST   /v1/workspaces/:workspaceId/archive
POST   /v1/workspaces/:workspaceId/restore
POST   /v1/workspaces/:workspaceId/transfer-ownership
```

Workspace context

```
POST /v1/workspaces/:workspaceId/select
GET  /v1/workspaces/:workspaceId/context
```

The backend should return:

```
{
  "workspace": {
    "id": "ws_123",
    "name": "Code X Stores",
    "status": "active"
  },
  "membership": {
    "role": "owner",
    "status": "active"
  },
  "products": [
    {
      "key": "inventory",
      "status": "active"
    }
  ],
  "permissions": [
    "workspace.view",
    "workspace.manage_members",
    "inventory.sell"
  ]
}
```

```
]
}
```

Membership endpoints

```
GET    /v1/workspaces/:workspaceId/members
GET    /v1/workspaces/:workspaceId/members/:membershipId
PATCH /v1/workspaces/:workspaceId/members/:membershipId
POST   /v1/workspaces/:workspaceId/members/:membershipId/suspend
POST   /v1/workspaces/:workspaceId/members/:membershipId/restore
DELETE /v1/workspaces/:workspaceId/members/:membershipId
```

Invitation endpoints

```
GET    /v1/invitations/:token
POST   /v1/invitations/:token/accept
POST   /v1/invitations/:token/decline
POST   /v1/workspaces/:workspaceId/invitations
GET    /v1/workspaces/:workspaceId/invitations
POST   /v1/workspaces/:workspaceId/invitations/:id/resent
POST   /v1/workspaces/:workspaceId/invitations/:id/revoke
```

Branch endpoints

```
GET    /v1/workspaces/:workspaceId/branches
POST   /v1/workspaces/:workspaceId/branches
PATCH /v1/workspaces/:workspaceId/branches/:branchId
DELETE /v1/workspaces/:workspaceId/branches/:branchId
```

Product access endpoints

```
GET    /v1/workspaces/:workspaceId/products
POST   /v1/workspaces/:workspaceId/products/:productKey/activate
POST   /v1/workspaces/:workspaceId/products/:productKey/deactivate
GET    /v1/workspaces/:workspaceId/products/:productKey/members
POST   /v1/workspaces/:workspaceId/products/:productKey/members
```

18. Example workspace service logic

Create workspace

```
authenticate user
validate request
check workspace creation limits
normalize workspace name
generate non-guessable workspace ID
```

```
generate unique slug
create workspace
create owner membership
create default roles
create selected product entitlements
create onboarding flow
write audit event
send welcome notification
return workspace context
```

Switch workspace

```
authenticate user
receive workspace ID
find membership using workspaceId + userId index
ensure membership status is active
ensure workspace status permits access
ensure requested product is enabled
create or update active workspace context
write workspace-selected event
return workspace and permissions
```

Invite member

```
authenticate user
resolve workspace
check workspace.manage_members permission
validate email and role
check existing membership
check duplicate pending invitation
generate secure token
store token hash
create invitation
write audit event
send invitation notification
return invitation status
```

Remove member

```
authenticate user
resolve workspace
check permission
find membership by composite workspaceId + membershipId
prevent removing only workspace owner
mark membership removed
revoke product memberships
revoke active sessions where required
write audit event
send notification
```

19. Notification plan

Workspace notifications

Owner

- Workspace created.
- Member joined.
- Member declined invitation.
- Member removed.
- Role changed.
- Ownership transferred.
- Product activated.
- Product suspended.
- Billing problem.

Invitee

- Workspace invitation received.
- Invitation expiring soon.
- Invitation accepted.
- Invitation revoked.

Member

- Added to workspace.
- Role changed.
- Product access granted.
- Product access revoked.
- Workspace archived.
- Workspace restored.

Notification channels

Initially:

- In-app.
- Email.

Later:

- Push.

- SMS.
- WhatsApp.

Notification examples

Subject: You have been invited to Code X Stores

Code X invited you to join Code X Stores as a Sales Attendant in Inventory.
Review invitation

Title: New member joined

Body: Mary Johnson accepted your invitation and joined Code X Stores as a Sales Attendant

20. Workspace audit events

Record:

```
workspace.created
workspace.updated
workspace.archived
workspace.restored
workspace.deletion_requested
workspace.deleted
workspace.ownership_transfer_started
workspace.ownership_transferred
workspace.member_invited
workspace.invitation_accepted
workspace.invitation_declined
workspace.invitation_revoked
workspace.member_role_changed
workspace.member_suspended
workspace.member_restored
workspace.member_removed
workspace.product_activated
workspace.product_deactivated
workspace.branch_created
workspace.branch_updated
workspace.branch_archived
workspace.workspace_selected
workspace.export_requested
workspace.support_access_granted
```

Audit requirements

- Append-only.
- Workspace-scoped.
- Request ID included.

- Actor included.
- Target entity included.
- Timestamp included.
- Severity included.
- Sensitive data excluded.
- Searchable by authorized administrators.

21. UI screen list

Application launcher

- Workspace list.
- Workspace selector.
- Application list.
- Product activation.
- Product access denied.
- Workspace creation.
- Join workspace.

Workspace settings

- Overview.
- General settings.
- Members.
- Invitations.
- Roles and permissions.
- Products.
- Branches.
- Billing.
- Security activity.
- Data export.
- Archive workspace.
- Delete workspace.

Member settings

- Member list.
- Member details.
- Change role.
- Change product access.
- Change branch access.
- Suspend member.
- Remove member.
- Activity.

Invitation settings

- Pending invitations.
- Accepted invitations.
- Expired invitations.
- Revoked invitations.
- Resend.
- Revoke.

22. Important user experience states

No workspace

You are not part of a workspace yet.
Create a workspace or accept an invitation to continue.

No product access

This workspace does not have access to Inventory.
Activate Inventory to begin managing products and sales.

Suspended workspace

This workspace is currently unavailable.
Contact the workspace owner or support for assistance.

Incomplete onboarding

Your Inventory workspace is not fully set up.
Continue setup to add products and start recording sales.

Multiple workspaces

Choose which workspace you want to open.

Removed membership

You no longer have access to this workspace.
Contact the workspace owner if you believe this is incorrect.

23. Billing and usage

Workspace-level billing is preferable to user-level billing for the business platform.

Subscription belongs to

workspace + product or workspace + plan

Depending on your commercial model:

Option A: Workspace-wide plan

One plan controls all activated products.

Option B: Product-specific plans

Each product has its own subscription.

Recommendation

Start with product-specific plans but share a central billing service.

Example:

Code X Stores

- Inventory Growth: Active
- Task Management Free: Active
- CRM: Not activated

Usage limits

Possible limits:

- Number of members.
- Number of branches.
- Number of products.
- Number of transactions.
- Number of workspaces.
- Storage.
- Export history.
- Product modules.

Usage should be checked in service logic, not only in the frontend.

24. Testing plan

Workspace creation

- Create workspace successfully.
- Duplicate request does not create duplicates.
- Required field validation.
- Slug collision.
- Unsupported workspace type.
- Owner membership creation.
- Default product entitlement.
- Audit event creation.

Workspace selection

- User with one workspace.
- User with multiple workspaces.
- User with no workspaces.
- User with suspended workspace.
- User without product access.
- Workspace switch refreshes permissions.
- Cross-workspace data blocked.

Invitations

- New user accepts invitation.
- Existing user accepts invitation.
- Expired invitation.
- Revoked invitation.
- Duplicate invitation.
- Wrong email.
- Declined invitation.
- Invitation role shown.
- Invitee receives access only after acceptance.

Memberships

- Change role.
- Suspend member.
- Restore member.
- Remove member.
- Cannot remove only owner.
- Removed member sessions revoked.
- Historical data preserved.

Ownership

- Transfer ownership.
- Invalid new owner.
- Re-authentication required.
- Notifications sent.
- Permissions updated.
- Audit generated.

Product access

- Activate product.
- Deactivate product.
- Trial expiry.
- Suspended product.
- User with no product membership.
- Correct product dashboard routing.

Security

- Forged workspace ID.
- Forged role.
- Cross-tenant object access.
- Insecure direct object reference.
- Unauthorized branch access.
- Session reuse after removal.
- Cache data leakage.
- Audit tampering.
- Rate-limit bypass.

Performance

- Workspace selector with many workspaces.
- Member list with many members.
- Invitation search.
- Product entitlement checks.
- Indexed Convex queries.
- Concurrent invitations.
- Concurrent membership updates.

25. Implementation phases

Phase 1: Data model and authorization foundation

Implement:

- Users.
- Workspaces.
- Memberships.
- Roles.
- Permissions.
- Product registry.
- Product entitlements.
- Branches.
- Convex schema.
- Required indexes.
- Request context.

Phase 2: Workspace creation

Implement:

- Create workspace screen.
- Workspace API.
- Owner membership.
- Default roles.
- Workspace settings.
- Creation notifications.
- Audit logs.

Phase 3: Workspace selector and switching

Implement:

- Workspace list.
- Product-aware filtering.
- Active workspace context.
- Workspace switch API.
- Secure permission refresh.
- Dashboard routing.

Phase 4: Invitations and members

Implement:

- Invite form.
- Email invitation.
- Invitation acceptance.
- Member list.
- Role changes.
- Suspension.
- Removal.
- Session revocation.
- Notifications and audit events.

Phase 5: Product access

Implement:

- Product registry.
- Product activation.
- Product entitlement.
- Product membership.
- Product-specific roles.
- Application launcher.
- Product access errors.

Phase 6: Branches

Implement:

- Branch creation.
- Branch management.
- Branch assignment.
- Branch-scoped permissions.
- Inventory branch selection.

Phase 7: Workspace lifecycle

Implement:

- Archive.
- Restore.
- Ownership transfer.
- Delete request.
- Deletion grace period.
- Workspace suspension.
- Billing state.

Phase 8: Hardening

Implement:

- Security testing.
- Cross-tenant testing.
- Audit review.
- Notification reliability.

- Rate limiting.
- Backup and recovery.
- Performance optimization.
- Support tools.

26. Release acceptance criteria

Workspace management is ready for MVP when:

- A user can create a workspace.
- The creator becomes owner.
- A workspace has no customer-specific subdomain.
- A user can belong to multiple workspaces.
- A user can select an active workspace.
- Workspace selection is validated server-side.
- Product access is workspace-specific.
- Users can activate Inventory.
- Users can invite members.
- Invitees can accept invitations.
- Roles can be assigned.
- Permissions are enforced by Fastify.
- Members can be suspended and removed.
- Removed members lose access.
- Historical records remain intact.
- Ownership transfer works safely.
- Workspace archive and restore work.
- Workspace deletion requires re-authentication and confirmation.
- Audit logs record important actions.
- Notifications are sent for important events.
- Convex queries use workspace-scoped indexes.
- Cross-workspace access is blocked.
- Branches can be created for Inventory.
- The architecture supports Task Management without creating another workspace system.

27. Final recommended structure

Use this hierarchy:

```
Orvionhub account
└─ User identity

Workspace
├─ Members
├─ Roles
├─ Permissions
├─ Branches
├─ Product entitlements
├─ Subscriptions
├─ Notifications
├─ Audit logs
└─ Product data

Product
├─ Inventory
├─ Task Management
├─ Gym
└─ Future products
```

The central rule is:

```
One user account
↓
Many workspaces
↓
Many products per workspace
↓
Different roles per product
↓
Workspace-scoped data and permissions
```

This model gives Orvionhub the flexibility of a Zoho-style platform without requiring organization-specific subdomains. It also ensures that Inventory can be launched first while Task Management and future products reuse the same workspace, membership, invitation, authorization, notification, and audit infrastructure.



1. https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html
2. <https://docs.convex.dev/database/reading-data/indexes/>
3. <https://docs.convex.dev/database/reading-data/>
4. <https://fastify.dev/docs/v5.0.x/Guides/Ecosystem/>
5. <https://github.com/fastify/fastify-auth/blob/main/README.md>
6. <https://docs.convex.dev/database/overview>

7. <https://fastify.dev/docs/latest/Guides/Ecosystem/>
8. <https://docs.convex.dev/api/classes/server.TableDefinition>
9. <https://fastify.dev/docs/v5.4.x/Guides/Ecosystem/>
10. <https://github.com/fastify/fastify-auth>
11. <https://github.com/fastify/fastify-bearer-auth>
12. <https://docs.convex.dev/database/reading-data/indexes/indexes-and-query-perf>
13. <https://fastify.dev/ecosystem/>
14. <https://fastify.dev/docs/v5.3.x/Guides/Ecosystem/>
15. <https://docs.convex.dev/database/schemas>